TranSPArent Reproduced Results Comparison

Table IV:

Paper:

TABLE IV: Accuracy breakdown of TRANSPARENT compared to baseline

Tool	FNR	FDR
TRANSPARENT	11/56 (19.6%)	24/57 (42.1%)
Vanilla CodeQL	35/56 (62.5%)	17/34 (50.0%)

Reproduced Results

```
---
Table 4                 FNR                     FDR
TranSPArent:            11/56 (19.6%);          24/57 (42.1%);
Vanilla CodeQL:         35/56 (62.5%);          17/34 (50.0%);
~/transparent-ae-1.0.0/accuracy
```

Observations and Conclusion:

Verdict: Exact Match

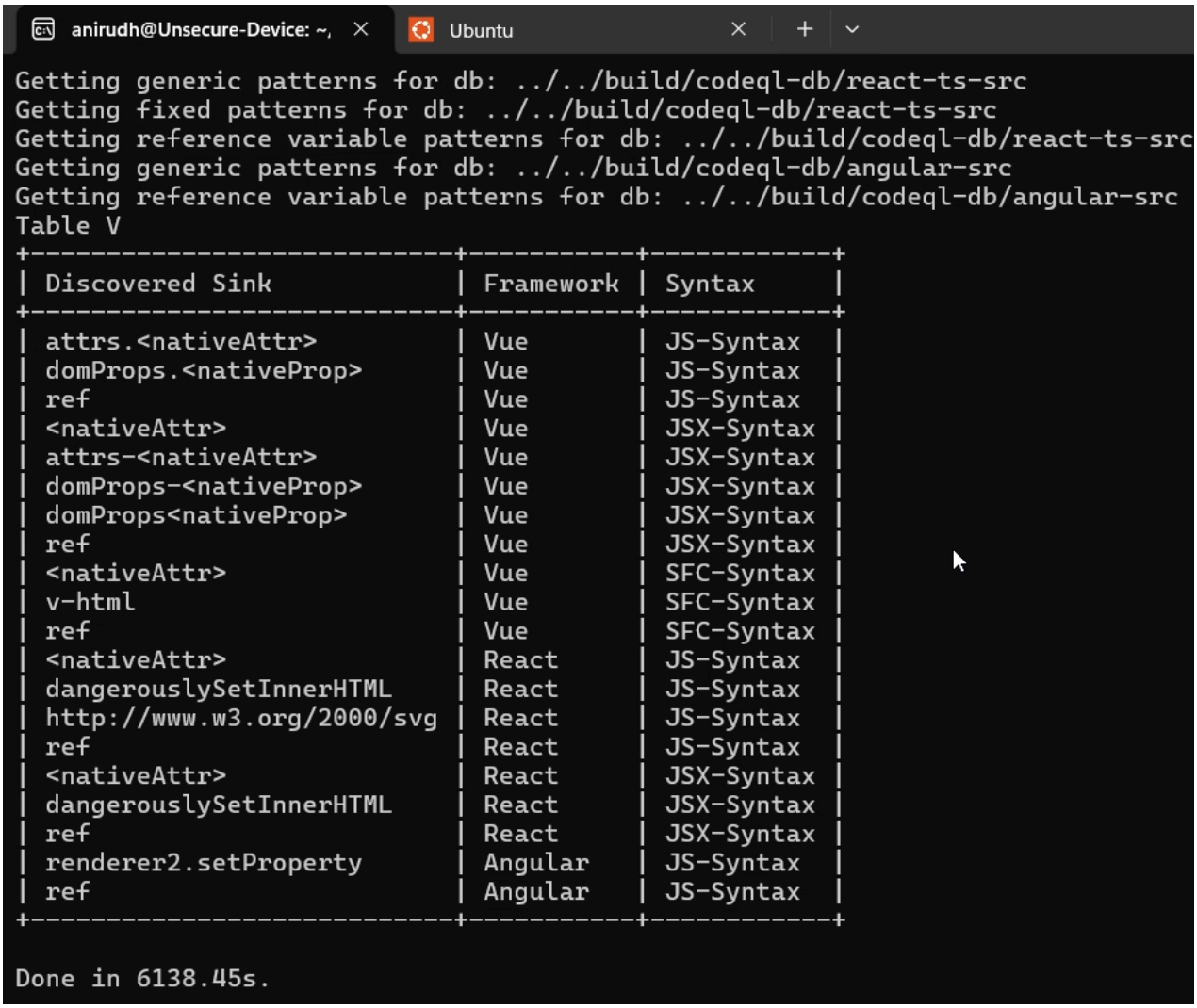
Table V:

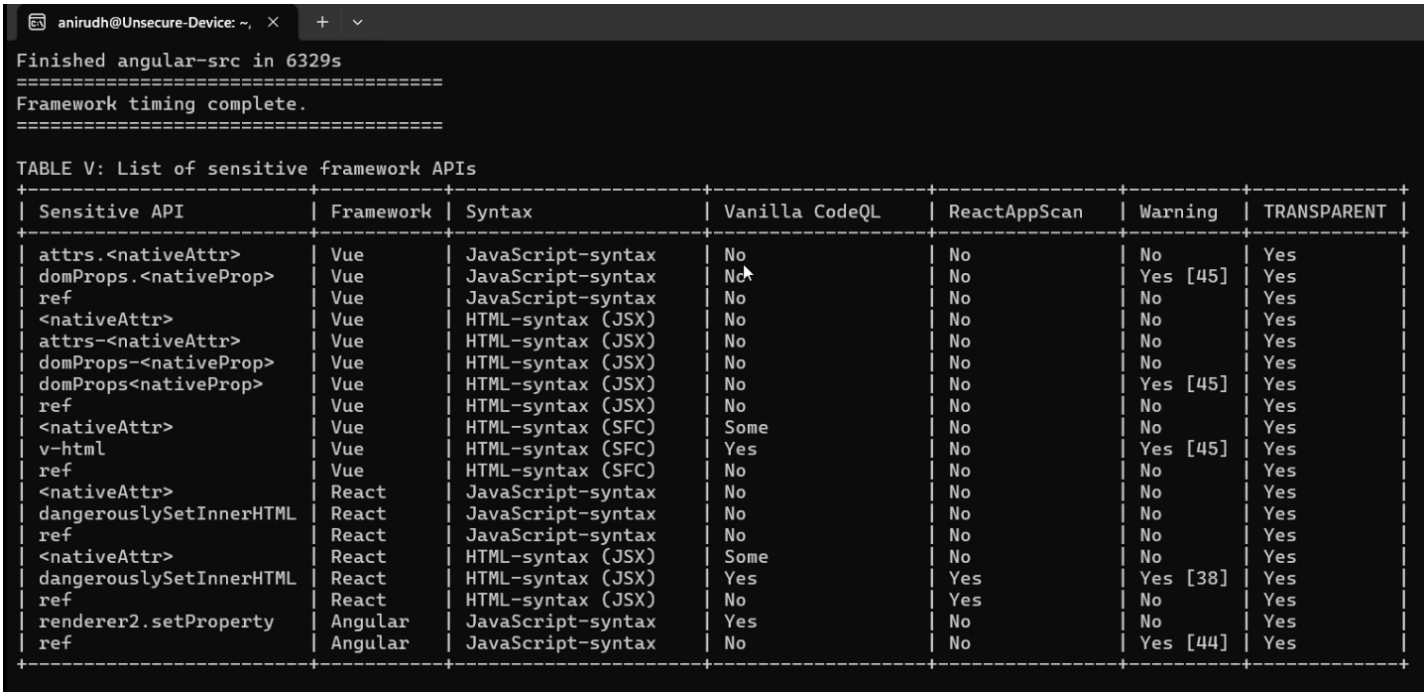
Paper:

TABLE V: List of sensitive framework API. `nativeAttr` and `nativeProps` correspond to any attributes and properties respectively of the underlying HTML tag. For list of sensitive attributes and properties, refer to Table II.

Sensitive Framework API	Framework	Syntax	Vanilla CodeQL	ReactAppScan	Warning	TRANSPARENT
attrs.<nativeAttr>	Vue	JavaScript-syntax	No	No	No	Yes
domProps.<nativeProp>	Vue	JavaScript-syntax	No	No	Yes [45]	Yes
ref	Vue	JavaScript-syntax	No	No	No	Yes
<nativeAttr>	Vue	HTML-syntax (JSX)	No	No	No	Yes
attrs-<nativeAttr>	Vue	HTML-syntax (JSX)	No	No	No	Yes
domProps-<nativeProp>	Vue	HTML-syntax (JSX)	No	No	No	Yes
domProps<nativeProp>	Vue	HTML-syntax (JSX)	No	No	Yes [45]	Yes
ref	Vue	HTML-syntax (JSX)	No	No	No	Yes
<nativeAttr>	Vue	HTML-syntax (SFC)	Some	No	No	Yes
v-html	Vue	HTML-syntax (SFC)	Yes	No	Yes [45]	Yes
ref	Vue	HTML-syntax (SFC)	No	No	No	Yes
<nativeAttr>	React	JavaScript-syntax	No	No	No	Yes
dangerouslySetInnerHtml	React	JavaScript-syntax	No	No	No	Yes
ref	React	JavaScript-syntax	No	No	No	Yes
<nativeAttr>	React	HTML-syntax (JSX)	Some	No	No	Yes
dangerouslySetInnerHtml	React	HTML-syntax (JSX)	Yes	Yes	Yes [38]	Yes
ref	React	HTML-syntax (JSX)	No	Yes	No	Yes
renderer2.setProperty	Angular	JavaScript-syntax	Yes	No	No	Yes
ref	Angular	JavaScript-syntax	No	No	Yes [44]	Yes

Reproduced Results





Sensitive API	Framework	Syntax	Vanilla CodeQL	ReactAppScan	Warning	TRANSPARENT
attrs.<nativeAttr>	Vue	JavaScript-syntax	No	No	No	Yes
domProps.<nativeProp>	Vue	JavaScript-syntax	No	No	Yes [45]	Yes
ref	Vue	JavaScript-syntax	No	No	No	Yes
<nativeAttr>	Vue	HTML-syntax (JSX)	No	No	No	Yes
attrs-<nativeAttr>	Vue	HTML-syntax (JSX)	No	No	No	Yes
domProps-<nativeProp>	Vue	HTML-syntax (JSX)	No	No	No	Yes
domProps<nativeProp>	Vue	HTML-syntax (JSX)	No	No	Yes [45]	Yes
ref	Vue	HTML-syntax (JSX)	No	No	No	Yes
<nativeAttr>	Vue	HTML-syntax (SFC)	Some	No	No	Yes
v-html	Vue	HTML-syntax (SFC)	Yes	No	Yes [45]	Yes
ref	Vue	HTML-syntax (SFC)	No	No	No	Yes
<nativeAttr>	React	JavaScript-syntax	No	No	No	Yes
dangerouslySetInnerHTML	React	JavaScript-syntax	No	No	No	Yes
ref	React	JavaScript-syntax	No	No	No	Yes
<nativeAttr>	React	HTML-syntax (JSX)	Some	No	No	Yes
dangerouslySetInnerHTML	React	HTML-syntax (JSX)	Yes	Yes	Yes [38]	Yes
ref	React	HTML-syntax (JSX)	No	Yes	No	Yes
renderer2.setProperty	Angular	JavaScript-syntax	Yes	No	No	Yes
ref	Angular	JavaScript-syntax	No	No	Yes [44]	Yes

Observations and Conclusion:

We consider the data up to the TRANSPARENT column as provided.

TranSPArent Column: Exact match. We also identified an additional React sink corresponding to the SVG namespace (https://www.w3.org/2000/svg). **This sink was not explicitly listed in the original evaluation.**

As SVG elements map directly to DOM APIs and support injection-relevant attributes, they act as valid taint sinks. This aligns with the paper's observation that hard-coded sink lists can miss framework-specific or less common DOM interfaces.

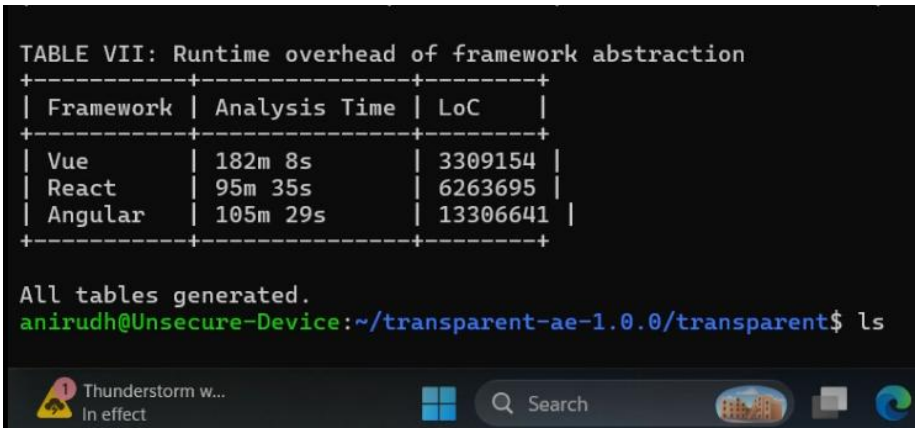
Table VII:

Paper:

TABLE VII: Runtime overhead of framework abstraction

Runtime	Analysis Time	LoC
Vue	57m	73k
React	1h 12m	353k
Angular	1h 15m	659k

Reproduced Result



Observations and Conclusion:

While the relative trend of analysis time across frameworks is broadly consistent with the original paper, our reproduced results show significantly higher absolute runtimes and substantially larger lines of code. This discrepancy is primarily due to differences in analysis scope, environment, and dataset size.

Fig 5:

Paper:

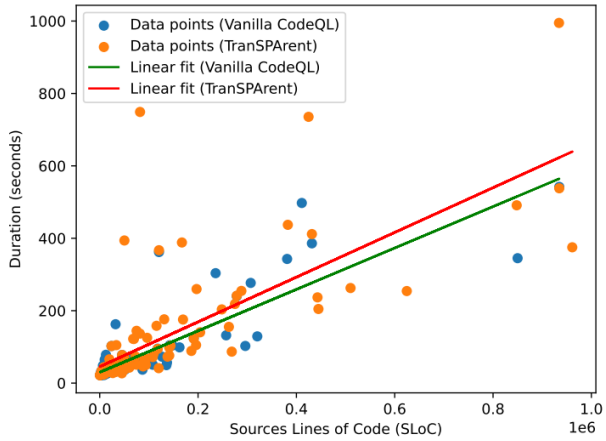
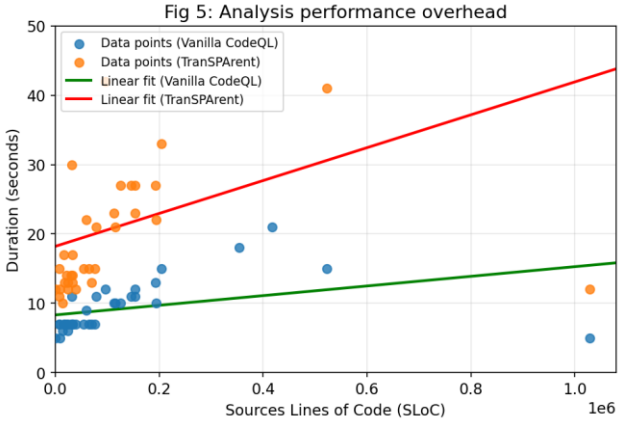


Fig. 5: Analysis performance overhead of 100 randomly sampled applications with respect to their line-of-code count.

Reproduced Results:



Observations and Conclusion:

- In both paper and reproduction:
  - Runtime increases with LoC (no surprise unless physics broke overnight)
  - TRANSPARENT (red) consistently lies above Vanilla CodeQL (green/blue)
- In the reproduction:
  - Much lower absolute durations (seconds scale vs hundreds)
  - Still shows clear linear trend
  - TRANSPARENT has a steeper slope → higher overhead per LoC

Conclusion:

Analysis time scales approximately linearly with codebase size. TRANSPARENT introduces a consistent overhead compared to Vanilla CodeQL, reflected in the steeper regression line. Although absolute runtimes differ from the original paper, the relative behavior and scaling trend are preserved.

Fig 6:

Paper:

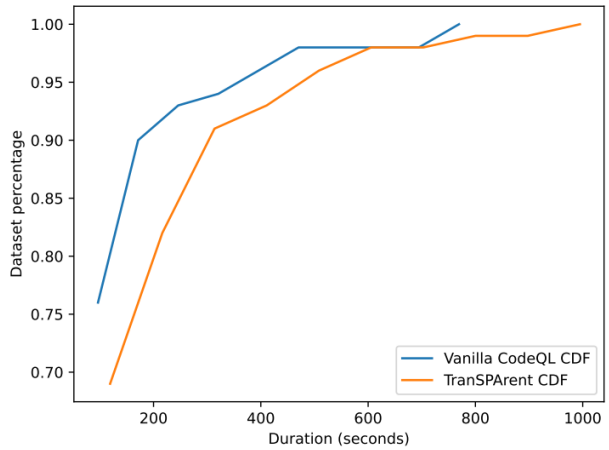
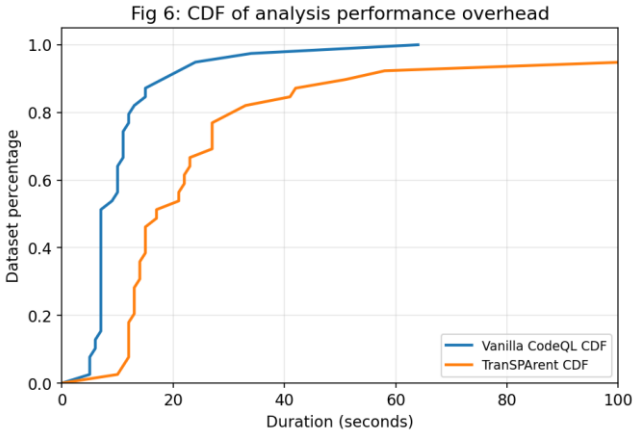


Fig. 6: CDF of analysis performance overhead of 100 randomly sampled applications.

Reproduced Results:



Observations and Conclusion:

- In both:
  - Vanilla CodeQL CDF rises faster → finishes earlier for most apps
  - TRANSPARENT curve is shifted right → slower overall
- In the reproduction:
  - Curves are compressed (lower durations overall)
  - But the gap between curves still exists

Conclusion:

The CDF confirms that TRANSPARENT incurs higher runtime overhead across the majority of applications. Vanilla CodeQL completes analysis faster for a larger fraction of programs, while TRANSPARENT exhibits a consistent rightward shift, indicating increased analysis cost.